Lab7 Handout : Use of yylval to communicate attributes of grammar symbols

The main objective of this handout is to explain how token attributes are passed from the scanner to the parser. The sharing of attributes is realized by assigning specific tasks to each tool, lex and yacc, and the scripts are written to ensure that the communication happens as desired by the user. We have already used two means of sharing that are reasserted below.

- Parser assigns distinct numbers, ≥ 258, to a symbol declared as a "token", and places them in the file "y.tab.h". Scanner includes this header file and passes the assigned numbers for the tokens, establishing communication for tokens between lex and yacc.
- The other means used is the scanner returning single character in quotes to the parser, and the grammar using the same form in the rules, which ensures desired communication between the s/w tools without any intermediary in between.

The 3rd means of communication, which is the main theme in this handout is by using a built-in variable, "yylval". This variable is defined in both the files, "y.tab.h" and "y.tab.c", the former for the scanner and the latter for the parser. Open up any one or both the files to find the lines of code given in column 1 of the following table.

#if ! defined YYSTYPE && ! defined YYSTYPE_IS_DECLARED typedef int YYSTYPE; # define YYSTYPE_IS_TRIVIAL 1 # define YYSTYPE_IS_DECLARED 1 extern YYSTYPE yylval;	The preprocessor directives state that if YYSTYPE is not defined and also YYSTYPE_IS_DECLARED is not defined, then a default definition for YYSTYPE is provided as of int data type. Further, yylval is defined as YYSTYPE, that is of int type as default.

Yacc permits the default "int" type of YYSTYPE (and hence of yylval) to be overridden by an user. This means of communication is illustrated though an example.

Problem : A string of comma separated elements terminated by a semicolon is given as input. An element is either a number (denoted by NUM) or an identifier (denoted by ID) as per rules of C. Write a parser that produces an output of the form shown below.

Input string	Generated Output
20, sum, count, 50, z;	Element 1 : Number is 20 Element 2 : Identifier is sum Element 3 : Identifier is count Element 4 : Number is 50 Element 5 : Identifier is z

Write a lex script and a yacc script to generate a parser that performs the desired task.

A simple grammar for the required langauge : S → cllist ‘;’ clist → clist ‘,’ elm | elm elm → NUM | ID

Tokens and attributes	Realization – several possibilities
Besides ‘;’ and ‘,’, the main tokens are NUM and ID. Each has 2 attributes : its type and the lexeme For NUM, we use code=1 for its type , and lexeme as its integer value For ID, we use code = 2 as type and lexeme as its name.	1. One way is that the scanner sets both the attributes and passes them to the parser. 2. Other way scanner sends the lexeme, value for NUM and name for ID to the parser. Parser sets the code, which indicates the type of token. Am elm is printed as soon it is seen by the parser. 3. Same as option 2 above, but instead of printing every elm as it is detected, each elm is appended to a list till all have been seen. Finally the list is used to print the elm objects one by one.

Design Option2 :

We illustrate option 2 and leave option 1 as an exercise for the reader. For the grammar rule of "elm" to work, scanner will continue to explicitly return NUM and ID. In addition, it has to pass the respective lexemes, an int or a char*. The latter is achieved by overloading the variable yylval, as illustrated below.

1. Note that YYSTYPE (or yylval) is a single object, and hence it can hold only one value at a time. However the "union" data type of C, permits many data types to coexist together.

2. The solution is therefore to define a union with all different types of tokens. The union is defined without any name in the yacc script, yacc automatically gives the name union as YYSTYPE. The terminal and nonterminal symbols of the grammar are assigned the appropriate type from the union. The syntax for doing this for the given problem is as follows.

Grammar rules in yacc notation	Attributes
start: clist ';' {} ; clist : clist ',' elm {} | elm {} ; elm : ID {} | NUM {} ; Nonterminals = {S, clist, elm}; 1st nonterminal is the start symbol. Terminals = {',' ';' NUM ID }	NUM : int value ID : identifier name elm : int value representing type, 1 for NUM and 2 for ID : value of NUM or name for ID clist : same 2 attributes as of elm
Attributes of yacc rules are accessed in a structured form as follows. The attribute for LHS symbol is referred to as "$$", while the symbols in the RHS in the listed order are referred to as "$1", "$2", and so on. The association for the rule, "clist : clist ',' elm", is explicated below. clist : clist ',' elm $$ $1 $2 $3 The $-variables of yacc are the attributes of the corresponding symbols in a rule. The number of attributes a $-variable has is decided by the writer of the rules as per the requirement of the problem.	
%union { char* idname; int ival; struct { int code; union { int val; char * name;} lexeme; }attr2; }	Defines YYSTYPE to be an union of three possible forms, i) char*, ii) int and iii) a struct with {int, int or char*}. Associate types to grammar symbols as follows. Token ID is to type char* is specified as "%token <idname> ID" NUM is of type int by writing "%token <ival> NUM" Both the nonterminals elm and clist are assigned a variable type by writing "%type <attr2> elm clist The types of all grammar symbols are defined in the yacc script

{digit}+
{yylval.ival = atoi(yytext); return NUM;}
{id}
{yylval.idname=(char*)malloc(sizeof(yytext));
strcpy(yylval.idname, yytext); return ID;
}
	The variable yylval, as defined in the union, is now used to communicate the attributes.
	For NUM, the variable "yylval.ival" is assigned the integer equivalent of the lexeme saved in yytext.
	For ID, the lexeme saved in yytext is copied to the variable, "yylval.idname"

The communication of attributes of both the tokens from the scanner to the parser have been set up properly. Actions against the grammar rules can be added now to solve the problem.

Consider the rule, elm : ID elm has 2 attributes, the lexeme for ID is available as $1, while code is to be assigned value 2 indicating it is token ID.	C code that implements evaluation of attributes follows. {$$.code = 2; $ $.lexeme.name = $1; elmcnt++; printf("Element %d : Identifier is %s\n", elmcnt, $$.lexeme.name); }
For the alternate rule of elm, elm : NUM	{$$.code = 1;$$.lexeme.val = $1; elmcnt++;

the attributes of elm are set similarly but we now assign, "$$.lexeme.val" to $1 to access its value	printf("Element %d : Number is %d \n", elmcnt, $$.lexeme.val);}

The design of both scripts are complete now in the files, "testlval1.l" and "testlval1.y" respectively.

Design Option 3 : The design decision is to create a list of elm objects as they are encountered during parsing and after entire input has been processed, produce the desired output by printing out each elm from the constructed list. A slightly different solution, as compared to Option 2, is adopted here. In the yacc script, the grammar remains unchanged but the attributes of the grammar symbols are different.

We first write C code for a list data structure, where each element of the list has 2 data items, an int and a char*. The decision here is to directly pass the lexeme both for NUM and ID. Basic list operations, for checking if a list in empty, find if a tuple comprising of an int and a char* is present in the list, append a tuple to a list and display a list. A simple design is given in the file, "list-functions.c" for you to test its working.

%union {
char* idname;
char* ival;
struct{int code; char*lexeme;}attr2;
struct node* list;
}
	idname and ival are both strings which the lexemes for the corresponding tokens, ID and NUM.
	The struct attr2 is a tuple of {int, char*} is defined to handle the nonterminal elm.
	For the symbol, clist, a list attribute is defined.

%token <idname> ID %token <ival> NUM %type <attr2> elm %type <list> clist	The association of attributes of terminal and nonterminal symbols are shown in the 1st column.

The main constituents of the scripts for lex and yacc are now completed using the design described above. Note how the 2 attributes of elm are assigned for ID and NUM differently. The list is constructed when the 1st elm is encountered during the rule, clist →elm, and the 2 attributes are added to the list associated with clist. The list is appended when the next elm is encountered in the rule, clist →clist ',' elm, when to the existing list denoted by $1, a new tuple associated with elm is added to update the existing list associated with $$.

{digit}+ {yylval.ival = (char*)malloc(sizeof(yytext));
strcpy(yylval.ival, yytext); return NUM;
}
 	
{id} {yylval.idname=(char*)malloc(sizeof(yytext));
strcpy(yylval.idname, yytext); return ID;
}
 	
Other regexes and actions are omitted.
	start: clist ';'
	{printf("Successful Parsing : %d elements found \n",
	elmcnt);
	display($1); }
	;
	clist : clist ',' elm
	{append(&$1,$3.code, $3.lexeme); $$=$1;}
	\| elm {if (elmcnt == 1) $$=mylist;
	append(&$$,$1.code, $1.lexeme); }
	;
	elm : ID {$$.lexeme = (char*)malloc(sizeof($1));
	$$.code=2; strcpy($$.lexeme,$1); ++elmcnt; }
	| NUM{$$.lexeme = (char*) malloc(sizeof($1));
	$$.code =1; strcpy($$.lexeme,$1); ++elmcnt;}
	;

The corresponding scripts are given in the files, "testlval2.l" and "testlval2.y" respectively.

End of Lab Handout 7